pro_ask_count_injection — review copy

the GPT-5.5 ask + the verbatim Isabelle definitions it ships with

Secretary working note — 2026-06-19

This PDF is for human review of the self-contained GPT ask in `posix-codex/pro_ask_count_injection/`. It reproduces, *verbatim and unedited*, the two files the external model receives: `PROMPT.txt` (the ask) and `DEFINITIONS.txt` (every function + cited green lemma, copied from the Isabelle source with file:line). The third attachment `CARD_FRONTIER.md` (the one-page state summary) is plain markdown and is not reproduced here.

1. The ask — `PROMPT.txt`

```
ROLE: Proof architect for an Isabelle/HOL cubic size-bound proof of a POSIX regex lexer. The WHOLE Gate is
    ↪  GREEN
(0 sorry) except ONE linear row-COUNT lemma, now pinned to a single cross-prune step. I need a CONCRETE,
Isabelle-formalizable lemma chain that closes it. Do not ask clarifying questions; design the proof (or
    ↪ prove no such
chain exists and characterize exactly what a correct proof must track).

EVERY function and cited lemma below is given VERBATIM (with file:line) in the attached DEFINITIONS.txt.
    ↪ Use those
exact definitions -- do NOT work from approximate/remembered semantics (a prior attempt failed precisely
    ↪ by mis-modeling
the collapsing tail). The state summary is in the attached CARD_FRONTIER.md.

================================================================================
0. PROTOTYPE & SELF-VALIDATE IN PYTHON BEFORE YOU ANSWER (mandatory)
================================================================================
You have a Python tool -- USE IT. Do NOT return a paper proof you have not run: the previous external
    ↪ attempt failed
precisely because its construction (a "collapsing-tail" ledger) was never tested and was false on a size
    ↪ -12 regex.
Procedure:
  1. Build a FAITHFUL Python model of the functions in DEFINITIONS.txt (use the EXACT equations). At
    ↪ minimum: the
     rrexp datatype (ZERO/ONE/CHAR/SEQ/ALTS/STAR suffice on the clean fragment); rsize; rsimp4_SEQ_atom (
    ↪ sigma4);
     rsimp7_SEQ_atom (sigma7 -- collapses ONLY a leading a*.a*); rflts; rdistinct; rsimp_ALTs; the prune
    ↪ chain
     (rprune_eq_against -> rsimpStrong_prune_pair_raw -> rsimpStrong_prune_against_rows_raw ->
     rsimpStrong_prune_rows_acc_raw -> rsimpStrong_prune_rows_raw); rsimpStrong_raw (S); rfrontier;
    ↪ row_dlforms (dl);
     rsimpStrong_dlform_closure; apder_terms; apder_frontier; apder_rows; apder_term_frontier_acc;
    ↪ strong_apder_acc;
     apder_strong_dlfrontier. SANITY-CHECK the model: card(apder_strong_dlfrontier r) <= rsize r + 1 must
    ↪ hold with
     0 violations, and the named CEs below must reproduce in your model. If your model disagrees, fix the
    ↪ model first.
  2. Encode your proposed injection / charge / invariant as Python and TEST it on: (a) exhaustive small
    ↪ regexes
     (rsize up to ~10); (b) the witness family -- nested SEQ(ALTS[pre,1], ... SEQ(atom, star*)) opened at
    ↪ k = star*,
     and stars-against-stars; (c) these named REGRESSION counterexamples (encode them and confirm your
    ↪ construction
     survives each):
       - SAA RALTS leak (rsize 7): r = ALTS[ONE, SEQ(STAR b, STAR a)], k = STAR a.
         (strong_apder_acc (ALTS rs) k keeps up to 4 uncollapsed ...(s*.s*) rows present in NO child.)
       - collapsing-tail over-approx CE (rsize 12):
         r = SEQ(ALTS[ONE, SEQ(ALTS[CHAR a, ONE], STAR a), CHAR a], STAR a)
```

```
        (U(r) contains the UNCOLLAPSED rows a*.a* and a.(a*.a*) -- a re-running-S tail loses them).
      - Tail2 unreachable-continuation leak: the SEQ step holds on REACHABLE k but leaks at standalone k
    ↪ = SEQ(STAR b, STAR b).
      - awidth refutation: r = ALTS[ONE, STAR(ALTS[STAR(CHAR c), ONE])] has card(U)=3 > awidth+1=2 (so do
    ↪  NOT target awidth+1).
  3. ITERATE: when your construction has a counterexample, find the SMALLEST one, diagnose, and REVISE --
    ↪ repeat until
    0 violations across (a)+(b)+(c) AND (for an injection) injectivity holds with 0 collisions.
  4. ONLY THEN write the Isabelle lemma chain. In your answer, REPORT the Python validation: the exact
    ↪ predicate(s)
    you tested, the sample space/size, that it found 0 counterexamples (incl. the named CEs), and any
    ↪ case your model
    could not cover. A construction you did not run in Python will be rejected.

================================================================================
1. THE OPEN LEMMA
================================================================================
  lemma card_apder_strong_dlfrontier_le:
    assumes "apder_clean r"
    shows   "card (apder_strong_dlfrontier r) \<le> Suc (rsize r)"     (* a LINEAR row COUNT *)
Write U(r) := apder_strong_dlfrontier r = rsimpStrong_dlform_closure (apder_rows r)
           = (UN q : apder_rows r. row_dlforms (rsimpStrong_raw q)).
Any LINEAR bound (card (U r) <= a * rsize r + b) also closes the Gate (loosen the downstream budget). A
    ↪ QUADRATIC
bound does NOT (it makes the Gate quartic). Discharging this lemma feeds
    ↪ actual_gate_from_direct_universe_rowlevel
(DEFINITIONS.txt §D) and the cubic Gate becomes unconditional. The bound is machine-validated TRUE (0
    ↪ violations,
tightest slack 0).

================================================================================
2. THE WALL (validated; do NOT re-propose a clean subset)
================================================================================
Three independent prover agents converged on this. The clean-subset induction is dead on BOTH natural
    ↪ carriers, each
leaking at exactly ONE constructor via the SAME a*.a* cross-prune (the SET-level rsimpStrong_ALTs_raw
    ↪ prune inside
rsimpStrong_raw keeps uncollapsed rows like ...(s*.s*) that a per-row S would collapse):
  * U(r) = apder_strong_dlfrontier r is RALTS-clean (apder_strong_dlfrontier_RALTS_subset, GREEN) but RSEQ
    ↪ /RSTAR-LEAKY.
  * SAA(r,k) = strong_apder_acc r k (the strong closure of the sigma4 carrier; it DISTRIBUTES over its
    ↪ carrier set,
    so RCHAR/RSEQ/RSTAR carrier splits are CLEAN SUBSETS -- see strong_apder_acc_{RSEQ,RSTAR,RCHAR}_subset
    ↪ , GREEN) but
    is RALTS-under-continuation LEAKY: strong_apder_acc (RALTS rs) k  is NOT  SUBSET of (UN q.
    ↪ strong_apder_acc q k)
    (smallest CE r=(1 + b*.a*), k=a*; the prune keeps up to 4 uncollapsed ...(s*.s*) rows present in no
    ↪ child).
So no single carrier closes by card_mono on a subset.

REFUTED routes (do NOT re-propose): the false dl(S q) SUBSET S‘dl q; U r SUBSET S‘Uun r (~56/66833 leak);
per-row size bounds (quartic); the deep frontier (U is NOT a subset of it); the awidth+1 injection (FALSE:
    ↪  r=(1+(c*+1)*)
gives card(U)=3 > awidth+1=2, every position key collides); the opened_boundary_forms transfer (it is
    ↪ sigma4 / NON-strong
-- see the GREEN refutation rsimpStrong_dlform_closure_opened_boundary_carrier_false in DEFINITIONS.txt §E
    ↪ ); the rewrite
route ->r' (does not commute with the derivative without the full ~1000-line Ch5 rewrite system).

================================================================================
3. WHAT I WANT (pick the SAA carrier; 3 of 4 constructor diff-steps already clean)
================================================================================
On the SAA carrier the diff-card induction card(strong_apder_acc r k - strong_apder_acc RONE k) <= rsize r
    ↪  has CLEAN,
already-written steps for RCHAR (card_strong_apder_acc_RCHAR_diff_base_le), RSEQ (via
    ↪ card_Un_Diff_telescope_le +
strong_apder_acc_RSEQ_subset + strong_apder_acc_RONE_sigma_subset), and RSTAR (telescope + a root-<=-1
    ↪ lemma). The ONE
```

```
remaining step is the LEAKY RALTS one, which has NO clean subset:

  GOAL STEP:  card( strong_apder_acc (RALTS rs) k  -  strong_apder_acc RONE k )
                <=  ( SUM_{q in set rs} card( strong_apder_acc q k  -  strong_apder_acc RONE k ) )  +  1.

The numeric inequality holds 0-violations (incl. the a*.a* killers); the obstruction is a DEFINABLE
    ↪ injection/charge,
NOT the count. Give EITHER:
  (a) a definable injection / transport LEDGER from the LHS prune-residual rows to the per-child RHS
    ↪ counts (+1 slack),
      operating at the BRANCH-REGEX level (transport a strong row to its SOURCE BRANCH under the same
    ↪ continuation,
      BEFORE opening -- analogous to a branch-origin lemma for rsimpStrong_ALTs_raw: every opened row of a
    ↪  SET-pruned
      strong alternation has a branch origin), accounting for the cross-pruned ...(s*.s*) survivors; OR
  (b) a "reachable clean-plug continuation" invariant P(k) that holds of every k the induction actually
    ↪ reaches
      (top k = RONE, then k := rsimp4_SEQ_atom r2 k / rsimp4_SEQ_atom (RSTAR r) k down the recursion) and
    ↪ is strong
      enough that the RALTS step holds under P(k) -- the leak is ONLY on UNREACHABLE standalone
    ↪ continuations
      (e.g. k = b*.b*), so a precise reachability invariant threaded through the induction would avoid it;
    ↪  OR
  (c) a proof that NO such injection/invariant is definable, with the minimal extra structure a correct
    ↪ proof must
      carry (a prune-order index? a provenance tag on rows?).

State every lemma as a precise Isabelle statement, naming the induction variable and the continuation
    ↪ parameter, and
say which DEFINITIONS.txt §D green facts each step uses. With the RALTS step, the induction closes, then
card(U r) <= Suc(rsize r) follows via apder_strong_dlfrontier_subset_strong_apder_acc_RONE +
    ↪ card_le_Suc_card_Diff_singleton,
and the Gate via actual_gate_from_direct_universe_rowlevel.

ATTACHMENTS: DEFINITIONS.txt (verbatim defs + green-lemma statements -- READ THIS), CARD_FRONTIER.md (
    ↪ state summary).
```

2. Verbatim definitions + green-lemma statements — DEFINITIONS.txt

```
VERBATIM ISABELLE DEFINITIONS + GREEN-LEMMA STATEMENTS
======================================================
Every function/lemma referenced by PROMPT.txt, copied verbatim from the source (file:line given).
Regex datatype rrexp constructors: RZERO, RONE, RCHAR c, RSEQ r1 r2, RALTS rs (rs :: rrexp list),
RSTAR r, RNTIMES r n, RBACKREF4 .., RHALF .., RRESIDUE .. . On the clean fragment (apder_clean) only
RZERO/RONE/RCHAR/RSEQ/RALTS/RSTAR occur (RNTIMES/backref-family excluded by rntimes_free + apder_nf).

OPAQUE (you do NOT need their internals): ‘afactored1 r s‘ and ‘rpder_strong_rows_raw c rs‘ are the lexer'
    ↪ s
derivative-rows functions; they appear ONLY in the final Gate corollary
    ↪ actual_gate_from_direct_universe_rowlevel
(§D), which is a GREEN black-box that consumes your count bound. Your target lemma
    ↪ card_apder_strong_dlfrontier_le
does NOT mention them. So treat them as opaque; prove only the count, and the Gate follows.

================================================================================================
A. CORE SIMPLIFIER / SEQUENCING FUNCTIONS
================================================================================================

(* base/BasicIdentities.thy:487 *)
fun rsize :: "rrexp \<Rightarrow> nat" where
  "rsize RZERO = 1"
| "rsize (RONE) = 1"
| "rsize (RCHAR  c) = 1"
| "rsize (RALTS  rs) = Suc (sum_list (map rsize rs))"
| "rsize (RSEQ  r1 r2) = Suc (rsize r1 + rsize r2)"
| "rsize (RSTAR  r) = Suc (rsize r)"
| "rsize (RNTIMES r n) = Suc (rsize r) + n"
| "rsize (RBACKREF4 r1 r2 r3 r4 cs) = Suc (rsize r1 + rsize r2 + rsize r3 + rsize r4)"
| "rsize (RHALF r cs rep) = Suc (rsize r)"
| "rsize (RRESIDUE cs rep) = 1"

(* base/BasicIdentities.thy:287  -- the WEAK sequence plug "sigma4" (NO a*.a* collapse) *)
fun rsimp4_SEQ_atom :: "rrexp \<Rightarrow> rrexp \<Rightarrow> rrexp" where
  "rsimp4_SEQ_atom RZERO r2 = RZERO"
| "rsimp4_SEQ_atom RONE r2 = r2"
| "rsimp4_SEQ_atom (RSEQ r1 r2) r3 = rsimp4_SEQ_atom r1 (rsimp4_SEQ_atom r2 r3)"
| "rsimp4_SEQ_atom (RCHAR c) RZERO = RZERO"
| "rsimp4_SEQ_atom (RCHAR c) RONE = RCHAR c"
| "rsimp4_SEQ_atom (RCHAR c) r2 = RSEQ (RCHAR c) r2"
| "rsimp4_SEQ_atom (RALTS rs) RZERO = RZERO"
| "rsimp4_SEQ_atom (RALTS rs) RONE = RALTS rs"
| "rsimp4_SEQ_atom (RALTS rs) r2 = RSEQ (RALTS rs) r2"
| "rsimp4_SEQ_atom (RSTAR r) RZERO = RZERO"
| "rsimp4_SEQ_atom (RSTAR r) RONE = RSTAR r"
| "rsimp4_SEQ_atom (RSTAR r) r2 = RSEQ (RSTAR r) r2"
| "rsimp4_SEQ_atom (RNTIMES r n) RZERO = RZERO"
| "rsimp4_SEQ_atom (RNTIMES r n) RONE = RNTIMES r n"
| "rsimp4_SEQ_atom (RNTIMES r n) r2 = RSEQ (RNTIMES r n) r2"
| "rsimp4_SEQ_atom (RBACKREF4 r1 r2 r3 r4 cs) RZERO = RZERO"
| "rsimp4_SEQ_atom (RBACKREF4 r1 r2 r3 r4 cs) RONE = RBACKREF4 r1 r2 r3 r4 cs"
| "rsimp4_SEQ_atom (RBACKREF4 r1 r2 r3 r4 cs) r5 = RSEQ (RBACKREF4 r1 r2 r3 r4 cs) r5"
| "rsimp4_SEQ_atom (RHALF r cs rep) RZERO = RZERO"
| "rsimp4_SEQ_atom (RHALF r cs rep) RONE = RHALF r cs rep"
| "rsimp4_SEQ_atom (RHALF r cs rep) r2 = RSEQ (RHALF r cs rep) r2"
| "rsimp4_SEQ_atom (RRESIDUE cs rep) RZERO = RZERO"
| "rsimp4_SEQ_atom (RRESIDUE cs rep) RONE = RRESIDUE cs rep"
| "rsimp4_SEQ_atom (RRESIDUE cs rep) r2 = RSEQ (RRESIDUE cs rep) r2"

(* base/BasicIdentities.thy:414  -- the STRONG plug "sigma7": sigma4 + top-head a*.a* -> a* collapse ONLY
    ↪ *)
definition rsimp7_SEQ_atom :: "rrexp \<Rightarrow> rrexp \<Rightarrow> rrexp" where
  "rsimp7_SEQ_atom r1 r2 =
    (case (r1, r2) of
      (RSTAR r, RSTAR s) \<Rightarrow> if r = s then RSTAR r else rsimp4_SEQ_atom r1 r2
    | (RSTAR r, RSEQ (RSTAR s) k) \<Rightarrow> if r = s then RSEQ (RSTAR r) k else rsimp4_SEQ_atom r1 r2
```

```
    | _ \<Rightarrow> rsimp4_SEQ_atom r1 r2)"

(* base/GeneralRegexBound.thy:18365  -- the strong normaliser "S". RSEQ via sigma7; RALTS via the SET-
    ↪ level
   rsimpStrong_ALTs_raw (the cross-row prune); RSTAR idempotent. *)
fun rsimpStrong_raw :: "rrexp \<Rightarrow> rrexp" where
  "rsimpStrong_raw RZERO = RZERO"
| "rsimpStrong_raw RONE = RONE"
| "rsimpStrong_raw (RCHAR c) = RCHAR c"
| "rsimpStrong_raw (RSEQ r1 r2) = rsimp7_SEQ_atom (rsimpStrong_raw r1) (rsimpStrong_raw r2)"
| "rsimpStrong_raw (RALTS rs) = rsimpStrong_ALTs_raw (rflts (map rsimpStrong_raw rs))"
| "rsimpStrong_raw (RSTAR r) =
    (case rsimpStrong_raw r of
      RZERO \<Rightarrow> RONE | RONE \<Rightarrow> RONE | RSTAR s \<Rightarrow> RSTAR s | s \<Rightarrow>
    ↪  RSTAR s)"
| "rsimpStrong_raw (RNTIMES r n) = RNTIMES (rsimpStrong_raw r) n"
| "rsimpStrong_raw (RBACKREF4 r1 r2 r3 r4 cs) = RBACKREF4 r1 r2 r3 r4 cs"
| "rsimpStrong_raw (RHALF r cs rep) = RHALF r cs rep"
| "rsimpStrong_raw (RRESIDUE cs rep) = RRESIDUE cs rep"

(* base/GeneralRegexBound.thy:18264  -- the SET-level alternation prune (rsimpStrong_prune_rows_raw is the
   cross-row pruning scan; THIS is the source of the "cross-prune keeps uncollapsed ...(s*.s*)" survivors)
    ↪ . *)
definition rsimpStrong_ALTs_raw :: "rrexp list \<Rightarrow> rrexp" where
  "rsimpStrong_ALTs_raw rs =
    rsimp_ALTs (rdistinct (rflts (rsimpStrong_prune_rows_raw rs)) {})"

(* base/BasicIdentities.thy:206 *)
fun rsimp_ALTs :: "rrexp list \<Rightarrow> rrexp" where
  "rsimp_ALTs  [] = RZERO"
| "rsimp_ALTs [r] =  r"
| "rsimp_ALTs rs = RALTS rs"

(* base/BasicIdentities.thy:177  -- flatten nested alternations, drop RZERO *)
fun rflts :: "rrexp list \<Rightarrow> rrexp list" where
  "rflts [] = []"
| "rflts (RZERO # rs) = rflts rs"
| "rflts ((RALTS rs1) # rs) = rs1 @ rflts rs"
| "rflts (r1 # rs) = r1 # rflts rs"

(* base/BasicIdentities.thy:83  -- dedup keeping first occurrence (acc = seen set) *)
fun rdistinct :: "'a list \<Rightarrow> 'a set \<Rightarrow> 'a list" where
  "rdistinct [] acc = []"
| "rdistinct (x#xs) acc = (if x \<in> acc then rdistinct xs acc else x # (rdistinct xs ({x} \<union> acc))
    ↪ )"

(* base/BasicIdentities.thy:25 *)
fun rnullable :: "rrexp \<Rightarrow> bool" where
  "rnullable (RZERO) = False"
| "rnullable (RONE) = True"
| "rnullable (RCHAR c) = False"
| "rnullable (RALTS rs) = (\<exists>r \<in> set rs. rnullable r)"
| "rnullable (RSEQ r1 r2) = (rnullable r1 \<and> rnullable r2)"
| "rnullable (RSTAR r) = True"
| "rnullable (RNTIMES r n) = (if n = 0 then True else rnullable r)"
| "rnullable (RBACKREF4 r1 r2 r3 r4 cs) = (rnullable r1 \<and> rnullable r2 \<and> rnullable r3 \<and>
    ↪ rnullable r4 \<and> cs = [])"
| "rnullable (RHALF r cs rep) = (rnullable r \<and> cs = [])"
| "rnullable (RRESIDUE cs rep) = (cs = [])"

-- the cross-row pruning scan used inside rsimpStrong_ALTs_raw, VERBATIM (5 functions, dependency order):

(* base/GeneralRegexBound.thy:17015  -- branch-set difference: keep the rrs-branches NOT in lrs (covered =
    ↪  lrs) *)
fun rprune_eq_against :: "rrexp list \<Rightarrow> rrexp list \<Rightarrow> rrexp list" where
  "rprune_eq_against covered [] = []"
| "rprune_eq_against covered (r # rs) =
    (if r \<in> set covered
```

```
     then rprune_eq_against covered rs
     else r # rprune_eq_against covered rs)"

(* base/GeneralRegexBound.thy:18163  -- the PAIRWISE rule: fires ONLY when both rows are (RALTS lrs).k1
    ↪ and
   (RALTS rrs).k2 with the SAME right factor k1 = k2; deletes from the later row's head-alternation the
    ↪ branches
   already in the earlier one, then re-plugs via sigma7. Every other pair leaves the LATER row unchanged.
    ↪ *)
definition rsimpStrong_prune_pair_raw :: "rrexp \<Rightarrow> rrexp \<Rightarrow> rrexp" where
  "rsimpStrong_prune_pair_raw earlier later =
    (case (earlier, later) of
      (RSEQ (RALTS lrs) k1, RSEQ (RALTS rrs) k2) \<Rightarrow>
        if k1 = k2
        then rsimp7_SEQ_atom (rsimp_ALTs (rprune_eq_against lrs rrs)) k2
        else later
    | _ \<Rightarrow> later)"

(* base/GeneralRegexBound.thy:18227  -- prune ONE row against ALL earlier rows (fold the pairwise rule
    ↪ over seen) *)
fun rsimpStrong_prune_against_rows_raw :: "rrexp list \<Rightarrow> rrexp \<Rightarrow> rrexp" where
  "rsimpStrong_prune_against_rows_raw [] r = r"
| "rsimpStrong_prune_against_rows_raw (x # xs) r =
    rsimpStrong_prune_against_rows_raw xs (rsimpStrong_prune_pair_raw x r)"

(* base/GeneralRegexBound.thy:18233  -- left-to-right scan; the accumulator 'seen' collects the PRUNED
    ↪ rows r'
   (NOT the originals), and rows are shrunk-never-dropped (length-preserving). *)
fun rsimpStrong_prune_rows_acc_raw :: "rrexp list \<Rightarrow> rrexp list \<Rightarrow> rrexp list" where
  "rsimpStrong_prune_rows_acc_raw seen [] = []"
| "rsimpStrong_prune_rows_acc_raw seen (r # rs) =
    (let r' = rsimpStrong_prune_against_rows_raw seen r
     in r' # rsimpStrong_prune_rows_acc_raw (r' # seen) rs)"

(* base/GeneralRegexBound.thy:18239  -- entry point (start with empty seen) *)
definition rsimpStrong_prune_rows_raw :: "rrexp list \<Rightarrow> rrexp list" where
  "rsimpStrong_prune_rows_raw rs = rsimpStrong_prune_rows_acc_raw [] rs"

(* KEY PROPERTY (why this is the wall): the pairwise rule only DELETES branches already present in an
    ↪ earlier row
   (rows shrunk, never dropped; language-preserving; size-non-increasing). The deletion is at the head-
    ↪ alternation
   SET level and the re-plug uses sigma7, which collapses ONLY a leading a*.a*. So a row of the form ...(s
    ↪ *.s*)
   whose head is NOT a leading a* can SURVIVE the prune UNCOLLAPSED -- this cross-prune residual has no
    ↪ per-row-S
   pre-image, which is exactly why U(SEQ ..)/SAA(RALTS ..) admit no clean subset. *)

================================================================================================
B. OPENING / FRONTIER / STRONG UNIVERSE
================================================================================================

(* base/GeneralRegexBound.thy:3862  -- the top-alternation split *)
fun rfrontier :: "rrexp \<Rightarrow> rrexp set"
  and rfrontiers :: "rrexp list \<Rightarrow> rrexp set" where
  "rfrontier RZERO = {}"
| "rfrontier (RALTS rs) = rfrontiers rs"
| "rfrontier r = {r}"
| "rfrontiers [] = {}"
| "rfrontiers (r # rs) = rfrontier r \<union> rfrontiers rs"

(* active/AntimirovFactoredTransition.thy:4791  -- "dl": open one regex to its linear rows.
   Distributes a top alternation; an alternation-headed sequence distributes its branches over the tail
   via sigma7 (the ONLY other place a*.a* can collapse); else bottoms out at the frontier. *)
function (sequential) row_dlforms :: "rrexp \<Rightarrow> rrexp set" where
  "row_dlforms RZERO = {}"
| "row_dlforms (RALTS rs) = (\<Union>q \<in> set rs. row_dlforms q)"
| "row_dlforms (RSEQ (RALTS ps) k) = (\<Union>p \<in> set ps. row_dlforms (rsimp7_SEQ_atom p k))"
```

```
| "row_dlforms r = rfrontier r"
  (* termination by measure rsize *)

(* active/AntimirovFactoredTransition.thy:5373 *)
definition row_dlformss :: "rrexp list \<Rightarrow> rrexp set" where
  "row_dlformss rs = (\<Union>q \<in> set rs. row_dlforms q)"

(* active/AntimirovFactoredTransition.thy:5376 *)
definition row_dlformss_set :: "rrexp set \<Rightarrow> rrexp set" where
  "row_dlformss_set U = (\<Union>q \<in> U. row_dlforms q)"

(* active/AntimirovFactoredTransition.thy:11335  -- the STRONG-opened closure of a set: open dl AFTER S.
   NB: S is applied INSIDE the union (per member), which is exactly why it DISTRIBUTES over its carrier
    ↪ set. *)
definition rsimpStrong_dlform_closure :: "rrexp set \<Rightarrow> rrexp set" where
  "rsimpStrong_dlform_closure U = (\<Union>p \<in> U. row_dlforms (rsimpStrong_raw p))"

(* active/AntimirovFactoredTransition.thy:11400  -- U(r), the target universe (over the deduped Antimirov
    ↪ rows) *)
definition apder_strong_dlfrontier :: "rrexp \<Rightarrow> rrexp set" where
  "apder_strong_dlfrontier r = rsimpStrong_dlform_closure (apder_rows r)"

================================================================================
C. ANTIMIROV STRUCTURES + REGIME PREDICATES
================================================================================

(* active/AntimirovFactoredTransition.thy:1715  -- Antimirov partial-derivative TERMS *)
fun apder_terms :: "rrexp \<Rightarrow> rrexp set" where
  "apder_terms RZERO = {}"
| "apder_terms RONE = {}"
| "apder_terms (RCHAR c) = {RONE}"
| "apder_terms (RALTS rs) = (\<Union>q \<in> set rs. apder_terms q)"
| "apder_terms (RSEQ r1 r2) = ((\<lambda>p. rsimp4_SEQ_atom p r2) ` apder_terms r1 \<union> apder_terms r2
    ↪ )"
| "apder_terms (RSTAR r) = ((\<lambda>p. rsimp4_SEQ_atom p (RSTAR r)) ` apder_terms r)"
| "apder_terms (RNTIMES r n) = (\<Union>m \<in> {..<n}. ((\<lambda>p. rsimp4_SEQ_atom p (RNTIMES r m)) `
    ↪ apder_terms r))"
| "apder_terms (RBACKREF4 r1 r2 r3 r4 cs) = {}"
| "apder_terms (RHALF r cs rep) = {}"
| "apder_terms (RRESIDUE cs rep) = {}"

(* active/AntimirovFactoredTransition.thy:1732 *)
definition apder_frontier :: "rrexp \<Rightarrow> rrexp set" where
  "apder_frontier r = rfrontier r \<union> (\<Union>q \<in> apder_terms r. rfrontier q)"

(* active/AntimirovFactoredTransition.thy:1736  -- the deduped Antimirov ROWS (index set of U(r)) *)
definition apder_rows :: "rrexp \<Rightarrow> rrexp set" where
  "apder_rows r = insert r (apder_frontier r)"

(* active/AntimirovFactoredTransition.thy:1739  -- the term-frontier ACCUMULATOR (continuation-parametric;
   the carrier strong_apder_acc is its strong closure). This is the D-law's accumulator. *)
fun apder_term_frontier_acc :: "rrexp \<Rightarrow> rrexp \<Rightarrow> rrexp set" where
  "apder_term_frontier_acc RZERO k = {}"
| "apder_term_frontier_acc RONE k = {}"
| "apder_term_frontier_acc (RCHAR c) k = rfrontier k"
| "apder_term_frontier_acc (RALTS rs) k = (\<Union>q \<in> set rs. apder_term_frontier_acc q k)"
| "apder_term_frontier_acc (RSEQ r1 r2) k =
    apder_term_frontier_acc r1 (rsimp4_SEQ_atom r2 k) \<union> apder_term_frontier_acc r2 k"
| "apder_term_frontier_acc (RSTAR r) k = apder_term_frontier_acc r (rsimp4_SEQ_atom (RSTAR r) k)"
| "apder_term_frontier_acc (RNTIMES r n) k =
    (\<Union>m \<in> {..<n}. apder_term_frontier_acc r (rsimp4_SEQ_atom (RNTIMES r m) k))"
| "apder_term_frontier_acc (RBACKREF4 r1 r2 r3 r4 cs) k = {}"
| "apder_term_frontier_acc (RHALF r cs rep) k = {}"
| "apder_term_frontier_acc (RRESIDUE cs rep) k = {}"

(* active/AntimirovFactoredTransition.thy:1779  -- alphabetic width (counts only RCHAR positions) *)
fun apder_awidth :: "rrexp \<Rightarrow> nat" where
  "apder_awidth RZERO = 0"
```

```
| "apder_awidth RONE = 0"
| "apder_awidth (RCHAR c) = 1"
| "apder_awidth (RALTS rs) = sum_list (map apder_awidth rs)"
| "apder_awidth (RSEQ r1 r2) = apder_awidth r1 + apder_awidth r2"
| "apder_awidth (RSTAR r) = apder_awidth r"
| "apder_awidth (RNTIMES r n) = n * apder_awidth r"
| "apder_awidth (RBACKREF4 r1 r2 r3 r4 cs) = 0"
| "apder_awidth (RHALF r cs rep) = 0"
| "apder_awidth (RRESIDUE cs rep) = 0"

(* active/AntimirovFactoredTransition.thy:1791  -- the structural normal form *)
fun apder_nf :: "rrexp \<Rightarrow> bool" where
  "apder_nf RZERO = True"
| "apder_nf RONE = True"
| "apder_nf (RCHAR c) = True"
| "apder_nf (RALTS rs) = (\<forall>q \<in> set rs. apder_nf q \<and> nonalt q \<and> q \<noteq> RZERO)"
| "apder_nf (RSEQ r1 r2) =
    (apder_nf r1 \<and> apder_nf r2 \<and> rnonseq r1 \<and> r1 \<noteq> RZERO \<and> r1 \<noteq> RONE \<
    ↪ and> r2 \<noteq> RZERO \<and> r2 \<noteq> RONE)"
| "apder_nf (RSTAR r) = apder_nf r"
| "apder_nf (RNTIMES r n) = apder_nf r"
| "apder_nf (RBACKREF4 r1 r2 r3 r4 cs) = True"
| "apder_nf (RHALF r cs rep) = True"
| "apder_nf (RRESIDUE cs rep) = True"

(* active/AntimirovFactoredTransition.thy:10394 *)
fun rntimes_free :: "rrexp \<Rightarrow> bool" where
  "rntimes_free RZERO = True"
| "rntimes_free RONE = True"
| "rntimes_free (RCHAR c) = True"
| "rntimes_free (RALTS rs) = (\<forall>q \<in> set rs. rntimes_free q)"
| "rntimes_free (RSEQ r1 r2) = (rntimes_free r1 \<and> rntimes_free r2)"
| "rntimes_free (RSTAR r) = rntimes_free r"
| "rntimes_free (RNTIMES r n) = False"
| "rntimes_free (RBACKREF4 r1 r2 r3 r4 cs) = True"
| "rntimes_free (RHALF r cs rep) = True"
| "rntimes_free (RRESIDUE cs rep) = True"

(* active/AntimirovFactoredTransition.thy:31587  -- the clean-fragment guard you may ASSUME on r. *)
definition apder_clean :: "rrexp \<Rightarrow> bool" where
  "apder_clean r \<longleftrightarrow>
    legacy_rrexp r \<and> rntimes_free r \<and> apder_nf r \<and> apder_zero_budget_trivial r"
(* legacy_rrexp = no backref/half/residue constructors; apder_zero_budget_trivial = a minor well-
    ↪ formedness
   flag. The two that matter for the count argument are rntimes_free (RNTIMES-free) and apder_nf (normal
    ↪ form).
   So you may freely assume r is built only from RZERO/RONE/RCHAR/RSEQ/RALTS/RSTAR, in apder_nf. *)

================================================================================
C2. AUXILIARY MEASURES & PREDICATES (referenced in the statements above/below)
================================================================================

(* active/AntimirovFactoredTransition.thy:380  -- ||X|| : opened-set SIZE = sum of rsize over a finite set
    ↪  *)
definition rsize_set :: "rrexp set \<Rightarrow> nat" where
  "rsize_set U = (\<Sum>q \<in> U. rsize q)"

(* base/BasicIdentities.thy:640  -- "not a top-level alternation" *)
fun nonalt :: "rrexp \<Rightarrow> bool" where
  "nonalt (RALTS  rs) = False"
| "nonalt r = True"

(* base/GeneralRegexBound.thy:3884  -- "not a top-level sequence" *)
fun rnonseq :: "rrexp \<Rightarrow> bool" where
  "rnonseq (RSEQ r1 r2) = False"
| "rnonseq r = True"
```

```
(* active/AntimirovFactoredTransition.thy:29681  -- odfront k = open the frontier of k (NON-strong); used
    ↪ in §E *)
definition odfront :: "rrexp \<Rightarrow> rrexp set" where
  "odfront k = row_dlformss_set (rfrontier k)"

(* base/GeneralRegexBound.thy:50  -- classical regexes (no backref/half/residue); component of apder_clean
    ↪  *)
fun legacy_rrexp :: "rrexp \<Rightarrow> bool" where
  "legacy_rrexp RZERO = True"
| "legacy_rrexp RONE = True"
| "legacy_rrexp (RCHAR c) = True"
| "legacy_rrexp (RALTS rs) = (\<forall>r \<in> set rs. legacy_rrexp r)"
| "legacy_rrexp (RSEQ r1 r2) = (legacy_rrexp r1 \<and> legacy_rrexp r2)"
| "legacy_rrexp (RSTAR r) = legacy_rrexp r"
| "legacy_rrexp (RNTIMES r n) = legacy_rrexp r"
| "legacy_rrexp (RBACKREF4 r1 r2 r3 r4 cs) = False"
| "legacy_rrexp (RHALF r cs rep) = False"
| "legacy_rrexp (RRESIDUE cs rep) = False"

(* active/AntimirovFactoredTransition.thy:29669  -- the "weight" = #letters + #stars. The GREEN root count
    ↪  is
   card(apder_rows r) <= apder_zw2 r + 2 <= rsize r + 2 (rntimes-free). *)
fun apder_zw2 :: "rrexp \<Rightarrow> nat" where
  "apder_zw2 RZERO = 0"
| "apder_zw2 RONE = 0"
| "apder_zw2 (RCHAR c) = 1"
| "apder_zw2 (RALTS rs) = sum_list (map apder_zw2 rs)"
| "apder_zw2 (RSEQ r1 r2) = apder_zw2 r1 + apder_zw2 r2"
| "apder_zw2 (RSTAR r) = Suc (apder_zw2 r)"
| "apder_zw2 (RNTIMES r n) = n * Suc (apder_zw2 r)"
| "apder_zw2 (RBACKREF4 r1 r2 r3 r4 cs) = 0"
| "apder_zw2 (RHALF r cs rep) = 0"
| "apder_zw2 (RRESIDUE cs rep) = 0"

(* active/AntimirovFactoredTransition.thy:31500  -- non-triviality flag (component of apder_clean): on the
    ↪  clean
   fragment it just forces every alternation to have positive weight. NOT relevant to the injection design
    ↪ . *)
fun apder_zero_budget_trivial :: "rrexp \<Rightarrow> bool" where
  "apder_zero_budget_trivial RZERO = True"
| "apder_zero_budget_trivial RONE = True"
| "apder_zero_budget_trivial (RCHAR c) = True"
| "apder_zero_budget_trivial (RALTS rs) =
    (apder_zw2 (RALTS rs) \<noteq> 0 \<and> (\<forall>q \<in> set rs. apder_zero_budget_trivial q))"
| "apder_zero_budget_trivial (RSEQ r1 r2) =
    (apder_zero_budget_trivial r1 \<and> apder_zero_budget_trivial r2)"
| "apder_zero_budget_trivial (RSTAR r) = apder_zero_budget_trivial r"
| "apder_zero_budget_trivial (RNTIMES r n) = False"
| "apder_zero_budget_trivial (RBACKREF4 r1 r2 r3 r4 cs) = False"
| "apder_zero_budget_trivial (RHALF r cs rep) = False"
| "apder_zero_budget_trivial (RRESIDUE cs rep) = False"

================================================================================================
D. THE CARRIER + GREEN LEMMAS TO CITE  (statements; all proved, no sorry)
================================================================================================

(* cubic/DirectUniverseCubic.thy:341  -- THE CARRIER. Note strong_apder_acc r k = the strong closure of
    ↪ the
   sigma4 carrier; because rsimpStrong_dlform_closure distributes over its set, the per-constructor
    ↪ carrier
   splits below are clean SUBSETS (RSEQ/RSTAR/RCHAR). The RALTS split is the open one. *)
definition strong_apder_acc :: "rrexp \<Rightarrow> rrexp \<Rightarrow> rrexp set" where
  "strong_apder_acc r k =
    rsimpStrong_dlform_closure (rfrontier (rsimp4_SEQ_atom r k) \<union> apder_term_frontier_acc r k)"

(* cubic/DirectUniverseCubic.thy:350  -- CLEAN RSEQ carrier split (GREEN) *)
lemma strong_apder_acc_RSEQ_subset:
  "strong_apder_acc (RSEQ r1 r2) k \<subseteq>
```

```
    strong_apder_acc r1 (rsimp4_SEQ_atom r2 k) \<union> strong_apder_acc r2 k"

(* cubic/DirectUniverseCubic.thy:370  -- CLEAN RSTAR carrier split (GREEN) *)
lemma strong_apder_acc_RSTAR_subset:
  "strong_apder_acc (RSTAR r) k \<subseteq>
    row_dlforms (rsimpStrong_raw (rsimp4_SEQ_atom (RSTAR r) k)) \<union>
    strong_apder_acc r (rsimp4_SEQ_atom (RSTAR r) k)"

(* cubic/DirectUniverseCubic.thy:377  -- CLEAN RCHAR carrier split (GREEN) *)
lemma strong_apder_acc_RCHAR_subset:
  "strong_apder_acc (RCHAR c) k \<subseteq>
    row_dlforms (rsimpStrong_raw (rsimp4_SEQ_atom (RCHAR c) k)) \<union>
    row_dlformss_set (rsimpStrong_raw ` rfrontier k)"

(* cubic/DirectUniverseCubic.thy:385 *)
lemma strong_apder_acc_RONE_RONE [simp]: "strong_apder_acc RONE RONE = {RONE}"

(* cubic/DirectUniverseCubic.thy:504  -- the continuation-base monotonicity used in the RSEQ telescope *)
lemma strong_apder_acc_RONE_sigma_subset:
  "strong_apder_acc RONE (rsimp4_SEQ_atom r k) \<subseteq> strong_apder_acc r k"

(* cubic/DirectUniverseCubic.thy:389  -- BRIDGE: U(r) sits inside the carrier at k = RONE (GREEN) *)
lemma apder_strong_dlfrontier_subset_strong_apder_acc_RONE:
  assumes "apder_nf r"
  shows "apder_strong_dlfrontier r \<subseteq> strong_apder_acc r RONE"

(* cubic/DirectUniverseCubic.thy:598  -- RCHAR diff-step base (GREEN): one new row over the base *)
lemma card_strong_apder_acc_RCHAR_diff_base_le:
  assumes "apder_nf k"
  shows "card (strong_apder_acc (RCHAR c) k - strong_apder_acc RONE k) \<le> rsize (RCHAR c)"

(* cubic/DirectUniverseCubic.thy:435  -- continuation rows fall into the RONE base (GREEN) *)
lemma row_dlforms_rsimpStrong_raw_subset_strong_apder_acc_RONE:
  assumes "apder_nf k"
  shows "row_dlforms (rsimpStrong_raw k) \<subseteq> strong_apder_acc RONE k"

(* cubic/DirectUniverseCubic.thy:633  -- the telescoping combinator (GREEN), core of RSEQ/RSTAR diff-steps
    ↪  *)
lemma card_Un_Diff_telescope_le:
  assumes "finite A" and "finite B" and "M \<subseteq> B"
  shows "card ((A \<union> B) - C) \<le> card (A - M) + card (B - C)"

(* cubic/DirectUniverseCubic.thy:307  -- EXCESS -> absolute lift (GREEN) *)
lemma card_le_Suc_card_Diff_singleton:
  assumes "finite A"
  shows "card A \<le> Suc (card (A - {x}))"

(* cubic/DirectUniverseCubic.thy:328 (GREEN) *)
lemma card_le_rsize_set:
  assumes "finite A"
  shows "card A \<le> rsize_set A"

(* active/AntimirovFactoredTransition.thy:19053  -- the CLEAN RALTS subset for the OTHER carrier U(r) (
    ↪ GREEN).
   (Useful contrast: U is RALTS-clean but RSEQ/RSTAR-leaky; strong_apder_acc is RSEQ/RSTAR/RCHAR-clean but
   RALTS-leaky. Neither is clean on all constructors -- the cross-prune wall.) *)
lemma apder_strong_dlfrontier_RALTS_subset:
  "apder_strong_dlfrontier (RALTS rs) \<subseteq> (\<Union>q \<in> set rs. apder_strong_dlfrontier q)"

(* active/AntimirovFactoredTransition.thy:37190  -- the linear ROOT count (GREEN) *)
lemma card_apder_rows_clean_le_rsize_plus_2:
  assumes "apder_clean r"
  shows "card (apder_rows r) \<le> rsize r + 2"

(* cubic/DirectUniverseCubic.thy:174  -- the GATE, GREEN modulo the CARD hypothesis. Discharge CARD and
    ↪ done. *)
lemma actual_gate_from_direct_universe_rowlevel:
  assumes "apder_clean r"
```

```
    and "card (apder_strong_dlfrontier r) \<le> Suc (rsize r)"
  shows "rsize_set (row_dlformss (rpder_strong_rows_raw c (afactored1 r s))) \<le> 2 * (rsize r + 3) ^ 3"
(* (per_row_size_le_quadratic @cubic:139 + universe_le_cubic_rowlevel @cubic:156 supply the rest; a looser
   LINEAR CARD bound card(U r) <= a*rsize r + b also closes the Gate after loosening
    ↪ budget_suc_quad_le_cube.) *)

================================================================================================
E. THE REFUTED SHORTCUT (so you don't re-propose it)
================================================================================================

(* active/AntimirovFactoredTransition.thy:29688  -- opened_boundary_forms opens with sigma4/rfrontier (NON
    ↪ -strong),
   NOT rsimpStrong_raw; odfront k = row_dlformss_set (rfrontier k). *)
definition opened_boundary_forms :: "rrexp \<Rightarrow> rrexp \<Rightarrow> rrexp set" where
  "opened_boundary_forms r k =
    row_dlformss_set (rfrontier (rsimp4_SEQ_atom r k) \<union> apder_term_frontier_acc r k) - odfront k"

(* active/AntimirovFactoredTransition.thy:32290  -- GREEN proof that U(r) does NOT embed into the non-
    ↪ strong
   opened_boundary_forms (so the green card bound on opened_boundary_forms does NOT transfer to U): *)
lemma rsimpStrong_dlform_closure_opened_boundary_carrier_false:
  fixes a :: char
  defines "r \<equiv> RSTAR (RALTS [RCHAR a])"
  defines "bad \<equiv> RSTAR (RCHAR a)"
  shows "apder_clean r"
    and "bad \<in> rsimpStrong_dlform_closure (set (afactored1 r []))"
    and "bad \<notin> odfront RONE \<union> opened_boundary_forms r RONE"
    and "\<not> rsimpStrong_dlform_closure (set (afactored1 r [])) \<subseteq>
      odfront RONE \<union> opened_boundary_forms r RONE"
```